Reading Notes: GeCCo

Guided Generation of Computational Cognitive Models

An Annotated Summary

Contents

Executive Summary

The GeCCo pipeline uses Large Language Models (LLMs) to automatically generate, fit, and iteratively refine computational cognitive models as Python functions. Given a task description, participant behavioural data, and a template function, GeCCo prompts an LLM to propose candidate models, fits those proposals to held-out data, and refines them based on feedback derived from predictive performance.

The approach was benchmarked across four cognitive domains—**decision making**, **learning**, **planning**, and **memory**—using three open-source LLMs of varying size, capacity, and training family. On all four human behavioural datasets, LLM-generated models consistently matched or outperformed the best domain-specific models from the literature, while remaining interpretable.

Code: https://github.com/MilenaCCNlab/gecco.git

Motivation

Computational cognitive models provide a formal framework for understanding human behaviour. In practice, researchers hand-craft these models, fit them to behavioural data, and iteratively refine them—a process that is slow, skill-intensive, and systematically biased. Translating a psychological theory into a working algorithm requires lengthy literature reviews and repeated trial-and-error coding cycles. Researchers must simultaneously act as domain experts, theoreticians, and software engineers—a rare combination. Perhaps most importantly, researchers tend to explore only the model classes they already know and favour, potentially missing superior explanations for the data.

This "streetlight effect" artificially shrinks the space of models considered, motivating an automated approach that is not anchored to any particular theoretical school.
Three specific LLM capabilities make them well-suited to cognitive model generation. First, LLMs can process task descriptions and behavioural data written in plain English, adapting across diverse experimental paradigms without a custom interface for each. Second, drawing on broad pre-training knowledge, they can identify structure in complex behavioural data and generate plausible psychological hypotheses. Third, LLMs can translate those hypotheses directly into executable Python functions, closing the loop from theory to testable model in a single step. These three abilities have already shown promise in automated statistical modelling through probabilistic program generation, and GeCCo applies the same logic to cognitive science.

GeCCo Pipeline

Each pipeline run consists of **10 sampling iterations**, repeated across **5 independent runs** per domain. On every iteration the LLM receives a structured prompt containing a natural language task description, behavioural data from a subset of participants (the *prompt* split), instructions and guardrails specifying the required Python function signature, a domain-specific template model for syntactic guidance, and—from iteration 2 onward—iterative feedback in the form of the current best-performing model and a list of previously used parameter names. Using this prompt, the LLM generates **three distinct cognitive models** with non-overlapping parameter sets.

Fitting and Evaluation

Generated models are parsed and converted to executable Python via `exec()`. Each model is then fit to a held-out *validation* dataset using `scipy.optimize.minimize` initialised from 20 random starting points. Model quality is scored with the **Bayesian Information Criterion (BIC)**—lower is better. The best-performing model is fed back into the next prompt.

Three separate data partitions prevent data leakage. The prompt data is seen by the LLM and provides context for hypothesis generation. The validation data is withheld from the LLM and used to score candidate models during the iterative loop. The test data is also withheld and reserved for the final evaluation against human baselines.

If a generated model contains syntax or runtime errors, the iteration restarts automatically—no human intervention required.

Metrics

Two complementary metrics are used throughout the evaluation. The **Bayesian Information Criterion (BIC)** balances goodness of fit against model complexity by penalising the number of free parameters; when two models fit equally well, BIC favours the simpler one. The pipeline is metric-agnostic, and AIC results are reported in Appendix I as a robustness check. The **Exceedance Probability (EXP)** is used in the final evaluation stage and quantifies the posterior probability that a given model provides the most prevalent explanation of observed behaviour across the population, translating raw BIC scores into an interpretable confidence measure.

Four canonical paradigms were selected to form an *escalating complexity* ladder—from static decisions to rich interactions between memory systems.

Experiment 1: Decision Making

Participants chose between two options defined by four features, each with a different validity (importance). The baseline was the probabilistic Weighted Additive Decision (pWADD) model, which uses four separate parameters to weight the four features. The Llama-generated model achieved a substantially lower BIC (39.40 vs. 51.97; $p < .001$). Rather than requiring four parameters, the LLM reduced the problem to a single *discount factor* that smoothly interpolates between three classic decision strategies—*Take the Best*, *Equal Weighting*, and *Weighted Additive*—mirroring complex Bayesian inference but arriving at the solution naturally.

Experiment 2: Learning

The task was a two-armed bandit with full feedback: after each trial participants also saw what the unchosen option would have paid, introducing regret and relief. The baseline was a four-learning-rate variant of the Rescorla–Wagner model ($RW_{4\alpha}$) augmented by a perseveration parameter capturing choice stickiness. The LLM-generated model (BIC 77.97 vs. 85.24; EXP = 0.97) replaced the asymmetric learning rates with a clean separation between actual and counterfactual value traces (*fictive trace*), and replaced

the perseveration parameter with a psychologically motivated *forgetting parameter* that decays unchosen-option values over time, mirroring the limits of working memory.

Experiment 3: Planning

The task was the classic two-stage Daw paradigm, in which first-stage choices lead to a second stage probabilistically, requiring forward planning. The baseline was a Hybrid model combining model-free (SARSA) and model-based value iteration, mixed by a free weighting parameter. The R1-generated model achieved a lower mean BIC (432.47 vs. 437.38; EXP = 0.85), although the BIC difference did not reach conventional significance ($p = .27$). By unifying the model-free/model-based dichotomy into a single system using two transition-dependent learning rates, and by omitting the temporal discounting parameter, the model aligned with recent theoretical calls to move beyond strict dual-system accounts of planning.

Experiment 4: Working Memory

Participants performed trial-and-error stimulus–response learning under varying cognitive load—either three or six associations held in mind simultaneously. The baseline was the RL-WM model (Collins & Frank, 2012), in which a slow RL module and a fast but capacity-limited WM module operate independently, with a load-dependent mixing weight. The Llama-generated model outperformed the baseline substantially (BIC 267.25 vs. 275.20; EXP = 0.99). Contrary to the standard assumption that reinforcement learning is immune to cognitive load, the LLM's model assigned separate RL learning rates for low- and high-load conditions (`lr_low` / `lr_high`), discovering that load modulates the core learning signal—a conclusion consistent with neuroimaging evidence on reward prediction error signals.

Control Experiments

A mixed-effects regression predicting BIC from *reasoning ability*, *model family*, and *model size* found that Llama outperformed Qwen ($\beta = 5.624$, $p = .03$) despite Qwen having more parameters, suggesting that training data and protocol matter more than raw scale. Reasoning ability showed only a marginal advantage ($\beta = -0.266$, $p = .07$), falling short of conventional significance.

Systematically removing each prompt component and refitting a mixed-effects model on post-ablation BIC scores revealed that all four components contribute meaningfully, though with different magnitudes. Iterative feedback was the strongest single driver of performance ($\beta = -17.040$, $p < .05$), followed by participant data ($\beta = -12.836$, $p < .05$), task description ($\beta = -9.834$, $p < .05$), and the code template ($\beta = -9.359$, $p < .05$). The dominance of the feedback component confirms that the iterative refinement loop—not merely the quality of the initial prompt—is what separates GeCCo from a single-shot query.

The LogProber method was applied to Llama prompts across all four domains. Acceleration scores were well below the contamination threshold of 1.0 (range: 0.017–0.648), confirming that the LLM is reasoning about novel problems rather than recalling memorised solutions from pretraining.

On synthetic datasets with known generating processes, the LLM successfully reverse-engineered the true model in both learning (Rescorla–Wagner) and decision-making (Take the Best; Tallying) domains, validating the mathematical soundness of the approach.

CENTAUR is a large foundation model trained to predict human behaviour across cognitive tasks, serving as a proxy for the ceiling of explainable variance. GeCCo matched or exceeded CENTAUR's negative log-likelihood across all four domains, while producing interpretable Python code rather than opaque model weights.

Discussion

We introduced GeCCo, an innovative pipeline that leverages open-source Large Language Models (LLMs) to automatically generate and iteratively refine computational cognitive models written as Python functions. When evaluated across four distinct cognitive domains, these LLM-generated models consistently matched or exceeded the performance of specialized baseline models. Through our control experiments, we discovered that the specific family of the underlying LLM significantly influences performance. Furthermore, while all elements of the prompt contributed to the pipeline's success, the iterative feedback mechanism proved to be the most crucial causal factor. We also confirmed that the prompts did not suffer from significant data leakage, and the pipeline successfully recovered ground-truth models from simulated datasets. Notably, the interpretable models produced by GeCCo outperformed the CENTAUR foundation model in predictive power, demonstrating that they capture the vast majority of explainable variance in human behavior.

The pursuit of automated model discovery has historically relied on handcrafted search algorithms and domain-specific languages to navigate restricted model spaces. However, recent breakthroughs in LLMs have provided new avenues to automate statistical modeling, generate hypotheses, and synthesize executable code. While other concurrent research has utilized evolutionary searches to build cognitive models, GeCCo distinguishes itself by offering a lightweight, open-source solution that operates entirely through in-context learning. This design choice allows for substantially faster convergence, reducing the necessary computational time from days to mere hours. Our approach capitalizes on the proven ability of LLMs to interpret natural language and accurately write general-purpose Python programs, bridging the gap between descriptive theories and executable models.

A primary strength of the GeCCo pipeline is its complete reliance on in-context learning using off-the-shelf, open-source LLMs without the need for any fine-tuning. This design significantly lowers the technical barrier to entry, empowering researchers to readily generate alternative hypotheses for their behavioral data. Additionally, GeCCo employs a hybrid optimization loop where the LLM solely functions as a proposal engine. Because candidate models are evaluated offline using traditional optimization tools, the model selection process remains strictly data-driven and safeguards against the LLM independently verifying its own outputs. The pipeline's demonstrated ability to generalize across various task domains further underscores its potential to help cognitive scientists efficiently explore a much wider and more diverse space of theoretical models.

Despite these promising results, our current methodology does have certain limitations. The pipeline utilizes a simple, domain-specific template within its prompt, which might anchor the LLM toward specific model classes. However, ablation studies indicated that

removing this template had a minimal effect on performance, and allowing the LLM to generate its own template yielded equally effective models. Another constraint is the pipeline's reliance on purely offline-computed numerical metrics, such as the Bayesian Information Criterion (BIC), for generating feedback. Future iterations could improve upon this by integrating a secondary LLM to act as a peer reviewer, offering qualitative feedback on theoretical plausibility, code quality, and parsimony, which could further minimize human oversight.

Furthermore, the domains we evaluated do not encompass the entirety of cognitive science. Expanding this approach to encompass perception, language processing, and high-dimensional data—such as vision—remains a complex but exciting frontier for future research. Generating models that operate with many parameters or within real-world, naturalistic datasets will be essential to proving the framework's broader utility. Ultimately, by democratizing access to sophisticated model discovery and removing the bottleneck of manual coding, LLMs hold the potential to dramatically accelerate the pace of research in computational cognitive science.

Reference

Milena et al. (2024). *GeCCo: Guided Generation of Computational Cognitive Models*. Available at: https://github.com/MilenaCCNlab/gecco.git